

Programando con Python

Pregunta 1

Crea una función que reciba los tres coeficientes a , b y c para resolver una ecuación de segundo grado. Muestra la solución por pantalla y ayúdate de la librería `math` para acceder a la función raíz cuadrada.

```
In [1]: import math

def ec2grado(a, b, c):
    discriminante = b**2 - 4*a*c
    if discriminante > 0:
        x_1 = (-b + math.sqrt(discriminante))/(2*a)
        print(x_1)
        x_2 = (-b - math.sqrt(discriminante))/(2*a)
        print("Ya que el discriminante es mayor que cero entonces la ecuación de segundo grado tiene dos raíces")
    elif discriminante == 0:
        x = -b/(2*a)
        print("Ya que el discriminante es cero entonces la ecuación de segundo grado tiene sólo una raíz real x")
    else:
        print("Ya que el discriminante es menor que cero entonces la ecuación no tiene resolución real.")

In [2]: ec2grado(4, 4, 1)
```

Pregunta 2

revés como por ejemplo "La ruta nos aportó otro paso natural").

```

vocales_con_tilde = {"Á": "a", "É": "e", "Í": "i", "Ó": "o", "Ú": "u"}
for key, value in vocales_con_tilde.items():
    s = s.replace(key, value) # reemplazo de vocales
    s = s.replace(" ", "") # quitar espacios
return s

def is_palindrom(s):
    for idx, c in enumerate(s):
        if s[idx] == s[len(s) - 1 - idx]:
            continue
        else:
            print("La frase NO es palíndroma")
            return # salir de la función
    print("La frase es palíndroma")

```

```

In [4]: frase = str(input("Escribe aquí tu frase:")).lower()
frase = string_cleaning(frase)

```

```

In [5]: is_palindrom(frase)

```

Pregunta 3

Crea un diccionario que tenga por claves los números del 1 al 10 y como valores sus raíces cuadradas.

```
In [6]: diccionario = {}

In [7]: import math

        for i in range(1, 11):
            diccionario[i] = math.sqrt(i)

In [8]: diccionario

Out[8]: {1: 1.0,
          2: 1.4142135623730951,
          3: 1.7320508075688772,
          4: 2.0,
          5: 2.23606797749979,
          6: 2.449489742783178,
          7: 2.6457513110645907,
          8: 2.8284271247461903,
          9: 3.0,
          10: 3.1622776601683795}
```

Pregunta 4

Crea un diccionario que tenga como claves las letras del alfabeto castellano y como valores los símbolos del código morse (los tienes todos en la Wikipedia). A continuación crea una función que lea una frase del teclado y te la convierta a morse utilizando el diccionario anterior.

```
In [9]: diccionario = {
    "A": ".-.",
    "B": "-...",
    "C": "-.-.",
    "CH": "----",
    "D": "-..",
    "E": ".-",
    "F": ".-.-",
    "G": "--.",
    "H": "...",
    "I": ".-.-",
    "J": "-.-.-",
    "K": "-.-.",
    "L": ".-.-.",
    "M": "--",
    "N": "-.-",
    "Ñ": "--.-.-",
    "O": "---",
    "P": ".-.-.",
    "Q": "--.-.",
    "R": "-.-.",
    "S": "...",
    "T": "--",
    "U": ".-.-",
    "V": ".-.-.-",
    "W": "-.-.-",
    "X": "-.-.-.",
    "Y": "-.-.-.",
    "Z": "-.-.-"
}

In [10]: s = str(input("Introduzca una frase:")).upper()

morse_list = []

for c in s:

    # si el caracter leído es un espacio guardamos un espacio en la lista cesar_list
    if c == " ":
        morse_list.append(" ")
        continue

    morse_list.append(diccionario[c])

frase_morse = " ".join(morse_list)
frase_morse

# Un espacio entre letras y tres espacios entre palabras
```

Pregunta 5

Créez une fonction qui prend les données des dictionnaires `nom` et `age` et les retourne en ordre.

```
In [11]: # Solución copiada de las soluciones porque entendí mal el enunciado
def clavesIguales(d1, d2):
    l = []
    for key in d1:
        if key in d2:
            l.append(key)
    return l

d1 = {"pera": 1, "manzana": 2, "cereza": 5, "sandia": 3}
d2 = {"manzana": 5, "melon": 1, "pera": 7, "melocoton": 4}

clavesIguales(d1, d2)

Out[11]: ['pera', 'manzana']
```

Pregunta 6

Crea una función que dado un número N nos diga si es primo o no (tiene que ir dividiendo por todos los números x comprendidos entre 2 y el número $N - 1$ y ver si la división de $\frac{N}{x}$ tiene resto cero o no).

```
[12]: def is_prime(N):  
        for x in range(2, N):  
            if N % x == 0:  
                print("El número NO es primo")  
                return  
        print("El número es primo")  
  
[13]: n = 11  
  
is_prime(n)
```

Pregunta 7

Investiga la documentación de la clase `string` y crea un método que lea una frase del teclado y escriba la primera letra de cada palabra

```
In [14]: def primera_letra_mayus(frase):  
         return frase.title() # utilizando el método title() se escribe la primera letra de cada palabra en mayúscula  
  
In [15]: frase = str(input("Escribe aquí tu frase:"))  
  
         primera_letra_mayus(frase)  
  
         'Estoy Dentro Del Curso De Estadística Descriptiva En Frogames Formación'
```

Pregunta 8

```
In [16]: n1 = int(input("Introduzca el primer número:"))
n2 = int(input("Introduzca el segundo número:"))

multiplos_1 = []
multiplos_2 = []

# calcular múltiplos del primer número
for mult in range(1, n1 + 1):
    if n1 % mult == 0:
        multiplos_1.append(mult)

# calcular múltiplos del segundo número
for mult in range(1, n2 + 1):
    if n2 % mult == 0:
        multiplos_2.append(mult)

# quedarnos con último múltiplo común (el más alto) de ambas listas
for i in multiplos_1:
    if i in multiplos_2:
        mcd = i

print("El máximo común divisor de {} y {} es {}".format(n1, n2, mcd))

El máximo común divisor de 12 y 16 es 4
```

Pregunta 9

Investiga el Círrado del Cesar y crea una función que lo reproduzca en Python. Cada letra del mensaje original se desplaza tres posiciones en el alfabeto estándar. La A se convierte en la D, la B se convierte en la E, la C se convierte en la F... y cuando se acaba el alfabeto se le vuelve a dar la vuelta: la X se convierte en la A, la Y en la B y la Z en la C. Los números no sufren ninguna modificación.

```
[17]: frase = str(input("Introduzca una frase:")).upper()

In [18]: def cifrado_cesar(n, s):
          cesar_list = []

          # si el caracter leído es un espacio guardamos un espacio en la lista cesar_list
          for c in s:
              if c == " ":
                  cesar_list.append(" ")
                  continue

              c_new = chr(ord(c) + n)
              if ord(c_new) > 90:
                  c_new = chr(ord(c_new) - 26)
              cesar_list.append(c_new)

          frase_cifrada = "".join(cesar_list)
          return frase_cifrada

In [19]: cifrado_cesar(3, frase)

Out[19]: 'HVWR HV XOD IUDVH HO HO FLIUDGR GHQ FHVDU'
```

Pregunta 10

Dada una lista de nombres de persona, escribe una función que los ordene de tres formas diferentes:

```
In [20]: lista_nombres = ["Jesús", "Guillermo", "Ana", "Carolina"]

A. De forma alfabética

In [21]: sorted(lista_nombres)

Out[21]: ['Ana', 'Carolina', 'Guillermo', 'Jesús']

B. De forma alfabética invertida

In [22]: sorted(lista_nombres, reverse=True)

Out[22]: ['Jesús', 'Guillermo', 'Carolina', 'Ana']
```

```
In [23]: sorted(lista_nombres, key=len)
```